Credit Scoring Card Combination Optimization Model Based on QUBO

Bole Zhang
Liangxin Honors College and College of Information Engineering, China Jiliang University
China
1437259673@qq.com

Weidi Wang
Liangxin Honors College and College of Information Engineering, China Jiliang University
China
2350897060@qq.com

Fang Cheng*
Liangxin Honors College and College of Materials and Chemistry, China Jiliang University
China
fangcheng@cjlu.edu.cn

Xiang Lin
Liangxin Honors College and College of Information Engineering, China Jiliang University
China
1460179693@qq.com

Abstract

This research focuses on the optimization of credit scoring models through the use of Quadratic Unconstrained Binary Optimization (QUBO) and heuristic algorithms, with the aim of refining decision-making processes in credit risk assessment. To maximize bank profitability while minimizing default risk, selecting different credit scoring cards and their threshold combinations is crucial. Using a QUBO model, the optimal credit scoring card and its thresholds were first identified using a quantum annealing algorithm. Subsequently, based on three credit scoring cards, the optimal threshold combination was sought using a genetic algorithm. Finally, among 100 credit scoring cards, the particle swarm optimization method was used to find the best combination of three cards, all aimed at maximizing the bank's ultimate revenue. This research provides objective and effective means for banks to reduce bad loan risks and achieve sustainable profitability.

CCS Concepts

• **Theory of computation** → Randomness, geometry and discrete structures; Random search heuristics.

Keywords

credit scoring card, combinatorial optimization model, QUBO model, quantum annealing algorithm, genetic algorithm, PSO particle swarm algorithm, quantum

ACM Reference Format:
Bole Zhang, Weidi Wang, Fang Cheng, and Xiang Lin. 2024. Credit Scoring Card Combination Optimization Model Based on QUBO. In *The International Conference on Machine Learning, Pattern Recognition and Automation Engineering (MLPRAE 2024), August 07–09, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3696687.3696715

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MLPRAE 2024, August 07–09, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0987-6/24/08
https://doi.org/10.1145/3696687.3696715

1 INTRODUCTION

In bank loan businesses, credit scoring cards are widely used to evaluate customers' credit status, thereby helping banks better control the risks associated with loan services. Due to the complexity of the banking context, it is often necessary to select a combination of multiple different credit scoring cards to achieve the best risk control strategy. Among the mathematical models for solving combinatorial optimization problems, models such as Integer Linear Programming (ILP) or Dynamic Programming (DP) can also address the issue, but they come with significant time overheads, high error probabilities, and are ineffective when dealing with large-scale data. To address this critical gap, our research introduces Quadratic Unconstrained Binary Optimization (QUBO)-based model for optimizing credit scoring card combinations. Due to its ability to run on quantum computer hardware, it achieves millisecond-level accelerated solutions through quantum computing. Thus, it provides a more efficient, reliable, and economical solution.

Prior to our research, several studies have been carried out in this area [1–3]. Despite providing certain insights into credit scores, these methods fell short in thoroughly optimizing credit scoring card combinations.

To solve optimization problems, adopting state-of-the-art algorithms is crucial. Traditional optimization methods, such as brute-force search, often face issues like long computation times, slow convergence rates, and a tendency to get stuck in local optima when dealing with large-scale, multi-dimensional complex problems. In contrast, heuristic algorithms can effectively enhance the efficiency and accuracy of the optimization process.

The methodology of this study is based on the integration of QUBO techniques [4], simulating the optimal combination of credit scoring cards using quantum annealing algorithm, genetic algorithm, and particle swarm optimization algorithm. By combining these algorithms, we can fully utilize their individual strengths, enhancing the efficiency, adaptability, and flexibility of the credit scoring model, thereby demonstrating high reliability and optimization effectiveness in various application scenarios.

Table 1: Two different credit scoring cards' each threshold corresponds to different default rates and approval rates.

Credit scoring card1			Credit scoring card2		
threshold	approval	default	threshold	approval	default
1	5%	0.50%	1	5%	0.50%
2	10%	1.00%	2	10%	1.00%
3	25%	1.50%	3	25%	1.50%
4	30%	2.00%	4	30%	2.00%
5	40%	2.50%	5	45%	2.50%
6	50%	3.00%	6	50%	2.70%
7	60%	3.50%	7	65%	3.50%
8	70%	4.00%	8	70%	4.00%
9	80%	4.50%	9	82%	4.70%
10	93%	5.00%	10	90%	5.00%

In the following sections, this research paper delves into the optimization of credit scorecard combinations based on QUBO. The study includes detailed data analysis and preprocessing, QUBO model formulation, heuristic algorithm solution, validation, and performance analysis. To conduct a qualitative analysis of the optimization of credit scorecard combinations, we meticulously collected a dataset of credit scorecards.

2 OVERVIEW OF DATA AND INTERPRETATION OF SYMBOLS

The data consists of 100 credit score cards, each of which can be configured with one of 10 thresholds. Here, t_1 represents the approval rate of credit score card 1 with 10 items, h_1 represents the default rate of credit score card 1 with 10 items, and so on. t_100 and h_100 represent the approval rate and default rate of credit score card 100, respectively.

Each credit score card has only one threshold is valid. Generally for banks, higher approval rates result in more customers passing the loan qualification assessment, thereby increasing the bank's interest income. However, a high approval rate usually corresponds to a high default rate, and defaults carry the risk of financial loss.

Table 1 below illustrates two different credit scoring cards, each with 10 thresholds, showing that each threshold corresponds to different default rates and approval rates.

Assuming the loan capital is 1,000,000 yuan and the bank's loan interest rate is 8%. If we choose a triple credit card combination strategy, the overall approval rate of this combination of credit scorecards is the product of all individual approval rates. The total bad debt rate is the average of the bad debt rates corresponding to the three credit score cards. The loan interest income is calculated by multiplying the loan capital by the interest income rate, the overall approval rate, and (1 - total bad debt rate). The bad debt loss caused by defaults is determined by multiplying the loan capital by the overall approval rate and the total bad debt rate. Finally, the bank's final income is obtained by subtracting the bad debt loss from the loan interest income.

In the optimal combination analysis, we continue to use the above-assumed conditions as the premise for the 100 sets of data and assume that the given pass rate and bad debt rate can accurately reflect real situations.

Table 2 gives a description of the symbols used below.

3 MODEL DEVELOPMENT AND SOLUTION APPROACH

In practice, finding the optimal single credit scoring card is the simplest and most common task. For this, we used the QUBO model and quantum annealing algorithms to solve the problem. While dynamic programming, linear programming, branch and bound, and exact algorithms are very effective for small-scale problems. However, for large-scale datasets (such as 1 million records), the computational complexity is extremely high and almost infeasible. In contrast, the QUBO model can effectively reduce computational complexity [5] and, utilizing quantum annealing algorithms, quickly find near-optimal solutions in exponential problems [6], thus improving time efficiency. First, a general planning model is established and then transformed into the QUBO form. The establishment of the general planning model is as follows:

Objective function:

$$max\ M = \sum_{i=1}^{n}\sum_{j=1}^{10} x_{i,j} \cdot \left(80000 \cdot p_{i,j}\left(1 - b_{i,j}\right) - 1000000 \cdot p_{i,j} \cdot b_{i,j}\right) \tag{1}$$

Constraints:
only one threshold can be selected per credit score card:

$$\sum_{j=1}^{10} x_{i,j} = 1, \forall i \in \{1, 2, ..., n\} \tag{2}$$

Each threshold can only be checked once:

$$\sum_{i=1}^{n} x_{i,j} = 1, \forall j \in \{1, 2, ..., 10\} \tag{3}$$

In the QUBO format, we need to transform the model into a quadratic unconstrained form, so we need to add constraints to the objective function [7]. Therefore, to address problems with constraints, penalty factors must be introduced into the objective function

First, we introduce a penalty factor a to satisfy the constraint that only one credit score card can be selected at a time. When a

Table 2: Description of symbols.

Symbol	Description	Unit
xi,j	Does the i-th credit score card select the threshold j	/
pi,j	Pass rate for the i-th credit score card corresponding to threshold j	%
bi,j	Bad debt rate for the i-th credit score card corresponding to threshold j	%
n	Number of credit score cards	/
M	Final revenue	RMB
a, β	Penalty factor	/
Q	Quadratic coefficient matrix	/
h	Vector of linear term coefficients	/
c	Constant term	/
C1	Inertial weighting	/
C2	Acceleration factor	/
w	Velocity direction	/
Mmax	Maximum profit	RMB
Emin	Smallest error	%
Emax	Maximum error	%

takes a very large positive number, the objective function will be very large if the constraint is not satisfied.

We choose the square term to take into account the nonnegativity of $x_{i,j}$, and the constraint takes the following form:

$$\alpha\left(\sum_{\mathrm{i}=1}^{100}\sum_{\mathrm{j}=1}^{10} x_{i,j} - 1\right)^2 \tag{4}$$

Secondly, we want to restrict $x_{i,j}$ to be 0 or 1 only, and not a small number of them, so we introduce a penalty factor β to constrain it.

We introduce a penalty factor β to constrain it, when β takes a very large positive number, if the constraint is not satisfied, the objective function will be very large, so as to satisfy the conditions of the constraint, the constraint is of the following form:

$$\beta\sum_{\mathrm{i}=1}^{100}\sum_{\mathrm{j}=1}^{10}\left(x_{i,j}^2 - x_{i,j}\right) \tag{5}$$

The objective function after adding these two terms can be expressed as:

$$\begin{aligned}\min M = &-1000000\cdot 0.08\cdot \sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot\left(1-b_{i,j}\right)\cdot x_{i,j}\\ &+1000000\cdot\sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot b_{i,j}\cdot x_{i,j}\\ &+\beta\textstyle\sum_{i=1}^{100}\sum_{j=1}^{10}(x_{i,j}^2 - x_{i,j}) + \alpha\left(\sum_{\mathrm{i}=1}^{100}\sum_{\mathrm{j}=1}^{10} x_{i,j} - 1\right)^2\end{aligned} \tag{6}$$

When converting an ordinary planning model (e.g., integer programming, Boolean programming, etc.) to a QUBO model, need to Separate the quadratic and linear terms in the objective function and constraints to obtain the form of the QUBO model:

$$\begin{aligned}\min M = &-1000000\cdot 0.08\cdot \sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot\left(1-b_{i,j}\right)\cdot x_{i,j}\\ &+1000000\cdot\sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot b_{i,j}\cdot x_{i,j}\\ &+\beta\textstyle\sum_{i=1}^{100}\sum_{j=1}^{10}(x_{i,j}^2 - x_{i,j}) + \alpha\left(\sum_{\mathrm{i}=1}^{100}\sum_{\mathrm{j}=1}^{10} x_{i,j} - 1\right)^2\end{aligned} \tag{7}$$

Now, the objective function and constraints are converted to the constrained form of the QUBO model:

$$\min M = x_{i,j}^T Q x_{i,j} + c \tag{8}$$

Where:

$$\begin{aligned}\mathrm{Q} = &-1000000\cdot 0.08\cdot \sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot\left(1-b_{i,j}\right)\cdot x_{i,j}\\ &+1000000\cdot\sum_{i=1}^{100}\textstyle\sum_{j=1}^{10} p_{i,j}\cdot b_{i,j}\end{aligned} \tag{9}$$

$$\mathrm{c} = \beta\sum_{i=1}^{100}\sum_{j=1}^{10}(x_{i,j}^4 - x_{i,j}^2)\alpha\left(\sum_{i=1}^{100}\sum_{j=1}^{10} x_{i,j}^2 - 1\right)^2 \tag{10}$$

This paper employs the quantum annealing algorithm to address the bank credit scorecard problem [8]. Advantage system5.3 and hybrid binary quadratic model 2 [9] are two different types of quantum annealers. The hybrid binary quadratic model version 2 hybrid quantum solver employs a hybrid approach combining classical and quantum computation to address quadratic unconstrained optimization (QUBO) problems [10]. The bank credit scorecard problem is encoded as a QUBO problem and solved using Hybrid Quantum Solver [11]. The specific steps for the solution are as follows [12]:

1. Initial state preparation: Initialize the quantum bits to a classical or quantum initial state, which serves as the starting point of the search.

2. Annealing process: By adjusting the parameters of the Hamiltonian, the quantum system evolves in the quantum state space and transitions from the initial state to the target state. In this process, the system may fall into the local optimal solution, but through quantum randomness and quantum entanglement, there is a chance to escape from the local optimal solution and continue to search for the global optimal solution.

3. Readout: After the annealing process, the final optimization result can be obtained by reading the states of the quantum bits.

In the algorithm, when converting the QUBO matrix into the BQM format and using the ExactSolver as the solver to find the optimal solution for the optimization problem, the time complexity

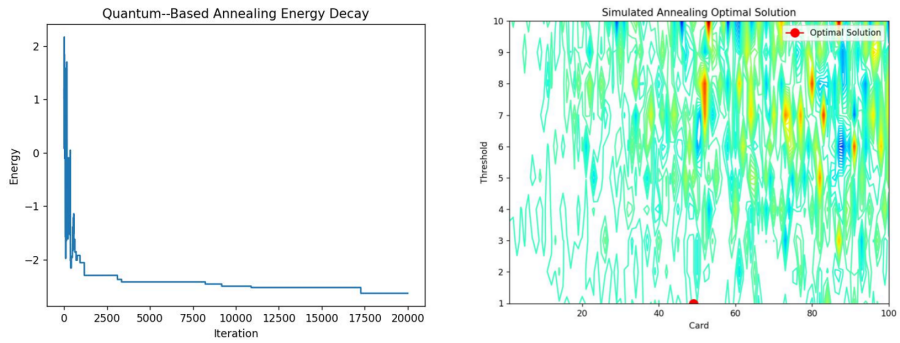

Figure 1: Energy decay diagram and two-dimensional planar desorption heat map.

Table 3: Table of parameters of the genetic algorithm

Parameters	Values
Population size	1000
Number of iterations	1000
Selection	roulette
Crossover	single-point crossover
Variation	site-specific mutation
Subpopulation replacement ratio	0.9

of this process includes the time complexity of constructing the QUBO matrix, which is O(num_cards * len(thresholds)^2). The time complexity of solving the optimization problem depends on the number of credit cards and the number of thresholds.

The final result obtained is as follows: the optimal threshold for credit score card 49 is 1 and the final revenue is 61172.00.

The energy decay graph in the quantum annealing algorithm and thermograms of different points in a two-dimensional plane are as follows:

Figure 1 indicates the optimal value (49,1) was finally found at 17219 iterations.

From Figure 1, it can be observed that the energy of quantum annealing continuously decreases and tends to stabilize, indicating good annealing effects. In the Two-dimensional Planar Desorption Heat Map, the most intense red spot located at (49,1) signifies the presence of a local optimum. This demonstrates that the annealing algorithm is highly suitable for optimization.

After addressing the issue of optimizing a single credit scorecard, the complexity significantly increases when we further study the optimal threshold combinations for the top three credit scorecards. To effectively tackle this increased complexity and find a satisfactory solution, we choose to employ a genetic algorithm.

The method of establishing a model and introducing a penalty factor is the same as before. Here, the constraint form of the QUBO model is given directly:

$$min\ M = x_{i,j}^T Q x_{i,j} + c \tag{11}$$

Where:

$$\begin{aligned} Q = &-1000000 \cdot 0.08 \cdot \textstyle\sum_{i=1}^{3} \sum_{j=1}^{10} p_{i,j} \cdot \left(1 - b_{i,j}\right) \\ &+1000000 \cdot \textstyle\sum_{i=1}^{3} \sum_{j=1}^{10} p_{i,j} \cdot b_{i,j} \end{aligned} \tag{12}$$

$$c = \beta \sum_{i=1}^{3} \sum_{j=1}^{10} \left(x_{i,j}^4 - x_{i.j}^2\right) + \alpha\left(\sum_{i=1}^{100} \sum_{j=1}^{10} x_{i,j}^2 - 3\right)^2 \tag{13}$$

The basic idea of a genetic algorithm is to generate a new set of solutions from the current solution space by simulating the genetic and evolutionary process in nature and to evaluate and select these solutions through the Fitness Function to generate the next generation of solutions [13]. After many generations of iterative evolution, the genetic algorithm gradually converges to the global optimal solution or local optimal solution.

The parameters of the problem model are listed in Table 3

In the algorithm, the code snippet reins(chrom{i},selch{i},1,1,objv{i},objvsel{i}) in Python is used to eliminate and replenish individuals while maintaining the population size. The time complexity of the algorithm depends on the population size and the number of generations, typically O(population size * number of individuals * number of variables^2 * number of generations).

The output of the results

Optimal solution: select the 1st, 2nd, and 3rd credit score cards with thresholds of 10, 10, and 10, respectively, and a maximum return of 20673.90.

The evolutionary iteration diagram is as follows:

From Figure 2, it can be seen that after seven iterations, the value of the objective function has stabilized, indicating the maximum profit.

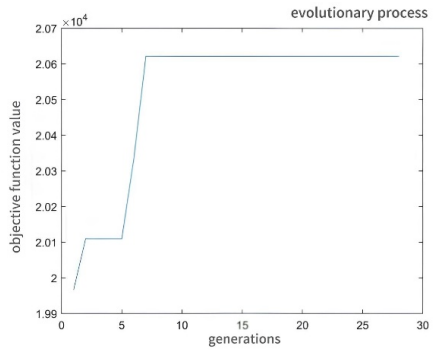

Figure 2: Evolutionary iteration chart.

Based on the completion of the first two steps, we can further refine the problem scenario to discuss the optimal combination of multiple credit scorecards. Due to the complexity of this credit scorecard combination optimization problem, we adopted the PSO algorithm.

Similarly, the method of model establishment and introducing penalty factors is the same as before. Here, the constraint form of the QUBO model is given directly:

$$min\ M = x_{i,j}^T Q x_{i,j} + c \tag{14}$$

Where:

$$\begin{aligned} Q = &-1000000 \cdot 0.08 \cdot \textstyle\sum_{i=1}^{100} \sum_{j=1}^{10} p_{i,j} \cdot \left(1 - b_{i,j}\right) \\ &+1000000 \cdot \textstyle\sum_{i=1}^{3} \sum_{j=1}^{10} p_{i,j} \cdot b_{i,j} \end{aligned} \tag{15}$$

$$c = \beta \sum_{i=1}^{100} \sum_{j=1}^{10} \left(x_{i,j}^4 - x_{i.j}^2\right) + \alpha \left(\sum_{i=1}^{100} \sum_{j=1}^{10} x_{i,j}^2 - 3 \right)^2 \tag{16}$$

The solution is then performed using the PSO particle swarm algorithm. Similar to genetic algorithms, PSO is also based on population iteration, but unlike genetic algorithms, PSO does not use crossover and mutation operations, but rather the particles follow the current optimal particles in the solution space [14]. The advantages of the PSO algorithm lie in its simplicity, ease of implementation, no need for gradient information, and few parameters, which make it suitable for dealing with real optimization problems.

In PSO, the positions and velocities of the particles can be seen as different combinations of credit scorecard thresholds. The fitness function can be linked to the objective function in QUBO, where our goal is to maximize (or minimize) the final gain. For this purpose, we choose three parameters c1, c2, w to help solve the problem where c1 represents the inertia weight, c2 represents the acceleration factor, and w represents the velocity direction.

In order to find the global optimal solution as much as possible, the parameters are selected as shown in Table 4

In particle swarm optimization (PSO) algorithm, the function pso_algorithm(qubo_matrix) in Python is used to call the PSO algorithm function and find the optimal solution and its corresponding fitness value. The time complexity of the algorithm depends on the number of iterations in the PSO algorithm and the size of the QUBO matrix, typically O (iterations x num_variables^2), where iterations is the number of iterations and num_variables is the number of variables.

Table 4: Particle swarm algorithm parameter list.

Parameters	Values
c1	0.35
c2	0.1
w	0.9

The process can be summarized in Figure 3

The final result is the selection of the 8th, 33rd, and 49th credit cards, the thresholds are 2, 6, and 3 respectively, and the final return is 43880.97.

The iterative optimization graph, Optimal Value Iteration Chart, and Particle Distribution at 100th Iteration are as follows:

From Figure 4, we find that as the number of iterations increases, the position fluctuations gradually decrease, indicating that the algorithm is approaching the global optimal solution, and the range of the perturbation is moderate, reducing the possibility of landing on the local optimal solution. Meanwhile, we can conclude that the Max_Profit optimal value gradually increases, which indicates that Max_Profit is seeking for the optimal solution. After a large number of iterations, the particles have already achieved their optimal positions and are relatively close in distance, which indicates that the algorithm has converged near a global optimal solution.

4 ANALYSIS AND IMPROVEMENT OF THE MODEL

First, we used the quantum annealing algorithm to solve the QUBO model. After multiple runs, the corresponding final revenues were 61050.00, 60930.00, 61080.00, and 60870.00. After solving the model, we compared the results with the results of the violence traversal and found that the results were: the optimal threshold for the credit score card 49 is 1, and the final revenue is 61172.00 [15]. Secondly, we used the genetic algorithm to solve the QUBO model, and compared the results with the traversal results. It was found that their thresholds were all 10, and the highest revenues were 20673.90 and 20673.00, respectively. Finally, in order to verify the effectiveness of the PSO algorithm in solving the QUBO model, the following model tests were performed: The model optimization is performed several times, and the results of each solution are averaged and compared with the best solution obtained by direct traversal.

In Table 5, Where M_{max} denotes the highest gain in ten iterations, E_{min} denotes the minimum error and E_{max} denotes the maximum error.

Based on the results of the data analysis, it can be concluded that the PSO algorithm performs well in the QUBO model and can be applied to practical problems such as combinatorial optimization and other fields.

The quantum annealing algorithm involves a large number of parameter settings and tuning, which may be challenging for users, and is non-deterministic, which may lead to randomness and unpredictability of the solution results, requiring multiple solving sessions to get better results.

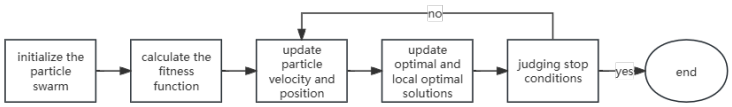

Figure 3: PSO flowchart.

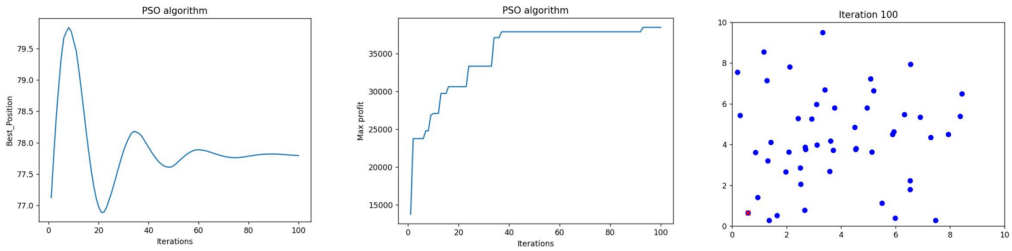

Figure 4: Iterative optimality seeking graph, optimal value iteration chart and particle distribution at 100th iteration.

Table 5: Comparison error.

Average maximum return	M_{max}	Average error	E_{min}	E_{max}
41113.09	43880.97	6.31%	0%	9.73%

Based on the QUBO heuristic algorithm, its result accuracy has been proven comparing with brute force solutions. Furthermore, Considering the algorithms that other banks can adopt, for example, Branch and Bound. It eliminates some choices that may not produce the best solution to save search time. However, in the worst-case scenario, it may need to traverse all the solutions. In comparison, heuristic algorithms can find a near-optimal solution within an acceptable time frame and can handle large-scale problems.

5 CONCLUSION

The proposed QUBO-based quantum annealing algorithm, genetic algorithm, and PSO algorithm in this article are all efficient optimization algorithms. They can select the most appropriate algorithm according to the characteristics of the problem and the data, quickly solve complex combinatorial optimization problems, and get the solution in a shorter time, thus improving efficiency. When banks or financial institutions do not have high requirements and only need to select for a single credit scorecard, but at the same time have a super large scale of data, i.e., a large number of credit scorecards, they can adopt the QUBO-based quantum annealing algorithm, which can save time and has high reliability. Although there is a negligible error in the final income prediction, there is almost no mistake in the choice of the credit card and its threshold. When it faces a large scale of data and needs to select multiple different credit scorecards to achieve the best risk control strategy, it can adopt the QUBO-based genetic algorithm and particle swarm algorithm given in the article. There are differences between the two. If the problem has complex and multi-peak characteristics, or involves discrete and combinatorial optimization problems, the genetic algorithm with its powerful global search capability and strong adaptability may be a better choice. For continuous optimization problems, or scenarios that need to quickly find a global optimal solution and are relatively simple to implement and tune, the particle swarm algorithm is more suitable. But both algorithms are equally accurate and reliable in choosing the credit scorecard and its threshold.

Acknowledgments

First and foremost, I would like to express my sincere gratitude to the reviewers for their invaluable feedback and insightful comments, which greatly improved the quality of this paper. I would also like to extend my gratitude to the editors for their guidance throughout the publication process. Furthermore, I would like to acknowledge the financial support received for this research. This work was supported by the Zhejiang Provincial Natural Science Foundation Major Project 2024 Provincial Basic Public Welfare Research Plan (LD24E01003), the Science and Technology Innovation Activity Program for College Students in Zhejiang Province (New Seedling Talent Program) Project (2024R409033), and the China Jiliang University 27th student research project (2024X27142). The generous support from these funding sources is greatly appreciated.

References

[1] Tripathi, D., *et al.* 2022. Credit scoring models using ensemble learning and classification approaches: a comprehensive survey. DOI: 10.1007/s11277-021-09158-9
[2] Xia, Y., *et al.* 2017. A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring. DOI: 10.1016/j.eswa.2017.02.017
[3] Zhao, Z., *et al.* 2015. Investigation and improvement of multi-layer perceptron neural networks for credit scoring. DOI: 10.1016/j.eswa.2014.12.006
[4] Du, Yu & Wang, Haibo & Hennig, Rick & Hulandageri, Amit & Kochenberger, Gary & Glover, Fred. 2023. New advances for quantum-inspired optimization. International Transactions in Operational Research. 10.1111/itor.13420.

[5] Glover, F., Kochenberger, G., Hennig, R. *et al.* 2022. Quantum bridge analytics I: a tutorial on formulating and using QUBO models. Ann Oper Res 314, 141–183. https://doi.org/10.1007/s10479-022-04634-2

[6] Glover, Fred & Lewis, Mark & Kochenberger, Gary. 2017. Logical and Inequality Implications for Reducing the Size and Complexity of Quadratic Unconstrained Binary Optimization Problems. European Journal of Operational Research. 10.1016/j.ejor.2017.08.025.

[7] Lewis, Mark & Glover, Fred. 2017. Quadratic Unconstrained Binary Optimization Problem Preprocessing: Theory and Empirical Analysis. Networks. 70. 10.1002/net.21751.

[8] Date, P., Arthur, D. & Pusey-Nazzaro, L. 2021. QUBO formulations for training machine learning models. Sci Rep 11, 10029. https://doi.org/10.1038/s41598-021-89461-4

[9] Zhou, T., Du, J., Sun, G., & Zhang, L. 2021. A hybrid algorithm based on QUBO model for credit scoring. International Journal of Computational Intelligence Systems, 14(1), 1222- 1232.

[10] S. Feng, J. Shi, R. Zhang, Y. Wang, F. Chen and X. Wang, "Optimization of Credit Score Card Portfolio Based on QUBO Model with Quantum Annealing Algorithm," 2023 IEEE International Conference on Electrical, Automation and Computer Engineering (ICEACE), Changchun, China, 2023, pp. 1187-1195, doi: 10.1109/ICEACE60673.2023.10441858.

[11] Pantelis Z. Lappas, Athanasios N. Yannacopoulos, A machine learning approach combining expert knowledge with genetic algorithms in feature selection for credit risk assessment, Applied Soft Computing, Volume 107, 2021, 107391, ISSN 1568-4946, https://doi.org/10.1016/j.asoc.2021.107391.

[12] L. Deng, P. Yang and W. Liu, "An Improved Genetic Algorithm," 2019 IEEE 5th International Conference on Computer and Communications (ICCC), Chengdu, China, 2019, pp. 47-51, doi: 10.1109/ICCC47050.2019.9064374.

[13] Zhu, S., & Zou, X. 2021. A novel genetic algorithm with a QUBO subproblem for mixed-integer programming. Journal of Heuristics, 27(3), 409-437.

[14] Ma, H., & Zhang, W. 2021. A novel binary PSO algorithm with dynamical adjustment for QUBO problems. Journal of Ambient Intelligence and Humanized Computing, 12(1), 145- 156.

[15] Zhang, X., & Liu, X. 2020. An improved genetic algorithm based on greedy search for QUBO problems. Journal of Physics: Conference Series, 1469(1), 012080.